To switch to using this headless driver, we have to modify the RSpec config we added in spec/rails_helper.rb to use this driver for our JavaScript system specs:

```
config.before(:each, type: :system, js: true) do
  driven_by :headless_selenium_chrome_in_container

  Capybara.server_host = "0.0.0.0"
  Capybara.server_port = 4000
  Capybara.app_host = 'http://web:4000'
end
```

Now if you rerun the system specs:

```
$ docker-compose exec web rspec spec/system/
```

you won't actually see the browser appear in the VNC window, but the test will pass.

Debugging

No chapter on testing would be complete without some mention of how to debug our application when it isn't behaving as we'd expect.

Imagine there's a problem with our welcome_controller.rb. Let's use the byebug debugger that's included as standard with a Rails app. Let's add a byebug breakpoint to our welcome_controller.rb:

```
class WelcomeController < ApplicationController
  def index
    redis = Redis.new(host: "redis", port: 6379)
    redis.incr "page hits"

    @page_hits = redis.get "page hits"
    byebug
  end
end
```

Let's stop our Rails server:

```
$ docker-compose stop web
```

When we want an *interactive* session with a container, rather than using docker-compose up, we instead use docker-compose run. Let's start an interactive session with our web server like this:

```
$ docker-compose run --service-ports web
```

By default, docker-compose run will ignore the port mappings specified in our docker-compose.yml file for the service. The --service-ports option changes this behavior and ensures they are mapped; without this, our Rails server wouldn't be accessible on port 3000 from the browser.